Taller — Análisis exploratorio de variables categóricas y numéricas

Soluciones y guía de facilitación

Dataset: `StudentsPerformance.csv`

Paso 0 — Preparación

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("StudentsPerformance.csv")
df.head()
```

Out[1]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

Parte 0 — Inspección inicial

Ejercicio 0.1 — Solución

In [2]:
```
print(df.shape)
print()
df.info()
print()
print("Nulos:", df.isna().sum().sum())
print("Duplicados:", df.duplicated().sum())
```

```
(1000, 8)

<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   str
 1   race/ethnicity               1000 non-null   str
 2   parental level of education  1000 non-null   str
 3   lunch                        1000 non-null   str
 4   test preparation course      1000 non-null   str
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), str(5)
memory usage: 62.6 KB

Nulos: 0
Duplicados: 0
```

> **Respuesta explicada**
>
> - `df.shape` = **(1000, 8)**: 1000 estudiantes y 8 columnas.
> - `info()` muestra 5 columnas de texto (`gender` , `race/ethnicity` , `parental level of education` , `lunch` , `test preparation course` ; aparecen como `object` o `str` según la versión de pandas) y 3 `int64` (las tres notas).
> - **0 nulos y 0 duplicados:** no hace falta limpiar nada. Aun así, el hábito de comprobarlo es el primer paso real de un EDA.
> - *Nota:* `race/ethnicity` contiene grupos anonimizados (A-E); es categórica, aunque se vea como una letra.

Respuesta esperada — Pregunta de reflexión 0: Categóricas: `gender` , `race/ethnicity` , `lunch` , `test preparation course` y `parental level of education` (esta última es **ordinal**: tiene un orden natural de `some high school` a `master's degree`). Numéricas: `math score` , `reading score` , `writing score` . No hay nada que limpiar porque no hay nulos ni duplicados y los tipos son los esperados; se sabe por `info()` (conteo de no nulos igual a 1000), `isna()` y `duplicated()` .

Parte 1 — Variables categóricas

Ejercicio 1.1 — Solución

In [3]:
```
print(df["gender"].value_counts())
print()
print(df["gender"].value_counts(normalize=True) * 100)
```

```
gender
female    518
male      482
Name: count, dtype: int64

gender
female    51.8
male      48.2
Name: proportion, dtype: float64
```

> **Respuesta explicada**
>
> - `value_counts()` cuenta cuántos estudiantes hay en cada categoría: **518** `female` y **482** `male`.
> - `value_counts(normalize=True)` devuelve proporciones (0 a 1); al multiplicar por 100 se obtiene el porcentaje: **51.8 %** y **48.2 %**.
> - Las dos tablas deben sumar 1000 y 100 %. Muestra casi balanceada por género.

Ejercicio 1.2 — Solución

In [4]:
```
print(df["test preparation course"].value_counts())
print()
print(df["test preparation course"].value_counts(normalize=True) * 100)
```

```
test preparation course
none         642
completed    358
Name: count, dtype: int64

test preparation course
none         64.2
completed    35.8
Name: proportion, dtype: float64
```

> **Respuesta explicada**
>
> - Mismo procedimiento con otra columna: **642** `none` **(64.2 %)** y **358** `completed` **(35.8 %)**.
> - Lo importante es que el estudiante reconozca que el código es reutilizable: solo cambia el nombre de la columna.
> - A diferencia de `gender`, aquí el desbalance es evidente (casi 2 de cada 3 no hicieron el curso).

Respuesta esperada — Pregunta de reflexión 1: `gender` está razonablemente balanceado (51.8 % `female` / 48.2 % `male`). `test preparation course` NO lo está: 64.2 % `none` (642) frente a 35.8 % `completed` (358). Esto importa porque, cuando un grupo es muy pequeño, las comparaciones de desempeño entre grupos son menos

confiables. Aquí el desbalance es moderado (358 casos siguen siendo suficientes para comparar), pero es el tipo de situación que hay que revisar antes de sacar conclusiones.

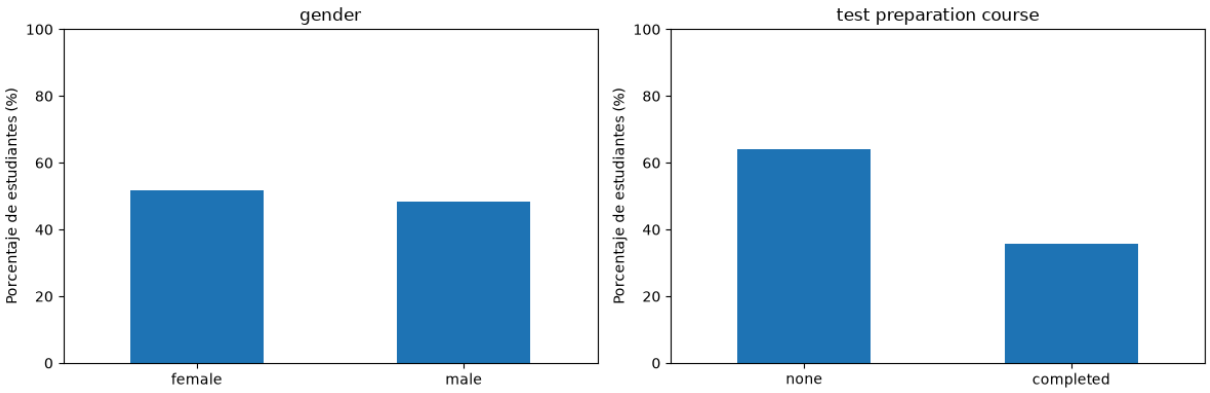
> *Punto de discusión en clase:* preguntar "¿qué pasaría si un grupo tuviera 950 estudiantes y el otro solo 50?" y contrastarlo con el caso real (642 contra 358): el problema crece cuando un grupo es muy pequeño, no por cualquier desbalance. Conecta con la idea de tamaño de muestra.

Ejercicio 1.3 — Solución

In [5]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, col in zip(axes, ["gender", "test preparation course"]):
    (df[col].value_counts(normalize=True) * 100).plot(kind="bar", ax=ax, rot=0)
    ax.set_title(col)
    ax.set_xlabel("")
    ax.set_ylabel("Porcentaje de estudiantes (%)")
    ax.set_ylim(0, 100)

plt.tight_layout()
plt.show()
```



> *Error común:* olvidar `ax=axes[i]`, lo que genera dos figuras separadas en vez de un panel de dos gráficos.
>
> *Detalle:* con `ylim(0, 100)` en ambos gráficos, el desbalance de `test preparation course` (64 % / 36 %) se ve de verdad frente al casi equilibrio de `gender` (52 % / 48 %). Con ejes automáticos las barras se verían parecidas.

Parte 2 — Variables numéricas

Ejercicio 2.1 — Solución

In [6]:
```
df["math score"].describe()
```

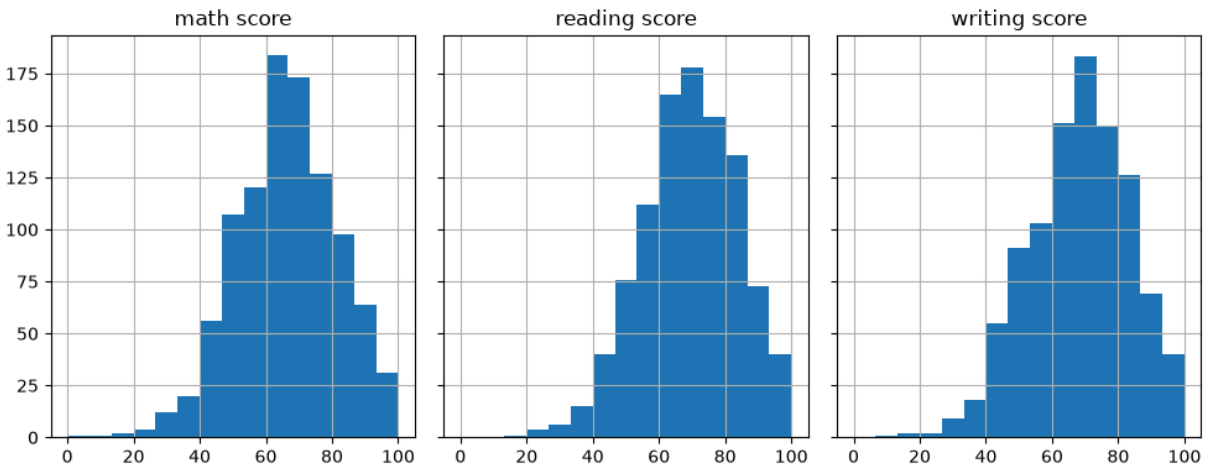
Out[6]:
```
count    1000.00000
mean       66.08900
std        15.16308
min         0.00000
25%        57.00000
50%        66.00000
75%        77.00000
max       100.00000
Name: math score, dtype: float64
```

> **Respuesta explicada**
>
> - `count` = 1000 (sin nulos), `mean` = **66.09**, `std` = **15.16** (qué tanto se alejan, en promedio, las notas de la media).
> - `min` = **0** y `max` = **100**; cuartiles: `25%` = 57, `50%` = 66 (mediana), `75%` = 77. El 50 % central de los estudiantes está entre 57 y 77.
> - **Error común:** leer `50%` como "el 50 % de los estudiantes": es la mediana, es decir, el valor que deja la mitad de las notas por debajo.

Respuesta esperada — Pregunta de reflexión 2: En `math score`, `mean` (66.09) y `50%` (66) son prácticamente iguales, lo que sugiere una distribución aproximadamente **simétrica**. Como contraste, en `reading score` (69.17 contra 70) y `writing score` (68.05 contra 69) la media queda algo por debajo de la mediana: es la pista de una cola leve hacia valores bajos (media < mediana).

Ejercicio 2.2 — Solución

In [7]:
```
df[["math score", "reading score", "writing score"]].hist(figsize=(10, 4), bins=15,
plt.tight_layout()
plt.show()
```



> **Respuesta explicada**

> - `.hist()` aplicado a las tres columnas dibuja un histograma por variable; `bins=15` agrupa las notas en 15 rangos.
> - Los tres tienen forma de campana centrada entre 66 y 69. Se aprecia una cola izquierda más larga que la derecha, porque las notas no pueden pasar de 100 pero sí hay estudiantes con notas muy bajas (el mínimo es 0 en matemáticas, 17 en lectura y 10 en escritura).
> - `range=(0, 100)` y `sharex/sharey` dejan los tres histogramas en la misma escala, así que se pueden comparar a simple vista (por defecto cada uno elige su propio eje y engaña).

Respuesta esperada — Pregunta de reflexión 3: Sí, las tres variables tienen forma aproximada de campana: pico central (alrededor de 60-75) y pocas barras en los extremos. Las tres muestran además una asimetría negativa leve y similar (coeficiente de asimetría ≈ −0.28, −0.26 y −0.29 para `math`, `reading` y `writing`): la cola izquierda es algo más larga porque hay estudiantes con notas muy bajas (mínimos de 0, 17 y 10) y el tope es 100. No es una asimetría marcada, pero sí es visible.

> *Sugerencia de facilitación:* mostrar el diagrama de campana visto en la sesión anterior y pedir a los estudiantes que señalen en su propio histograma dónde estaría "la media/mediana" y dónde las "colas".

Ejercicio 2.3 — Solución

In [8]:
```
df.describe()
```

Out[8]:

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

Respuesta esperada — Pregunta de reflexión 4: Con la salida de `df.describe()`:

	`math score`	`reading score`	`writing score`
`std`	15.16	14.60	15.20

	`math score`	`reading score`	`writing score`
`min`	0	17	10
`max`	100	100	100
Rango (`max` − `min`)	**100**	83	90

- **Mayor dispersión (`std`):** `writing score` (15.20), por muy poco sobre `math score` (15.16); `reading score` es la más concentrada (14.60). Las tres son muy parecidas, así que conviene que el estudiante no sobreinterprete una diferencia de décimas.
- **Rango más amplio:** `math score` (de 0 a 100, rango 100), por el estudiante con nota 0.
- Las medias son 66.09, 69.17 y 68.05: `reading` y `writing` rinden algo mejor que `math` .

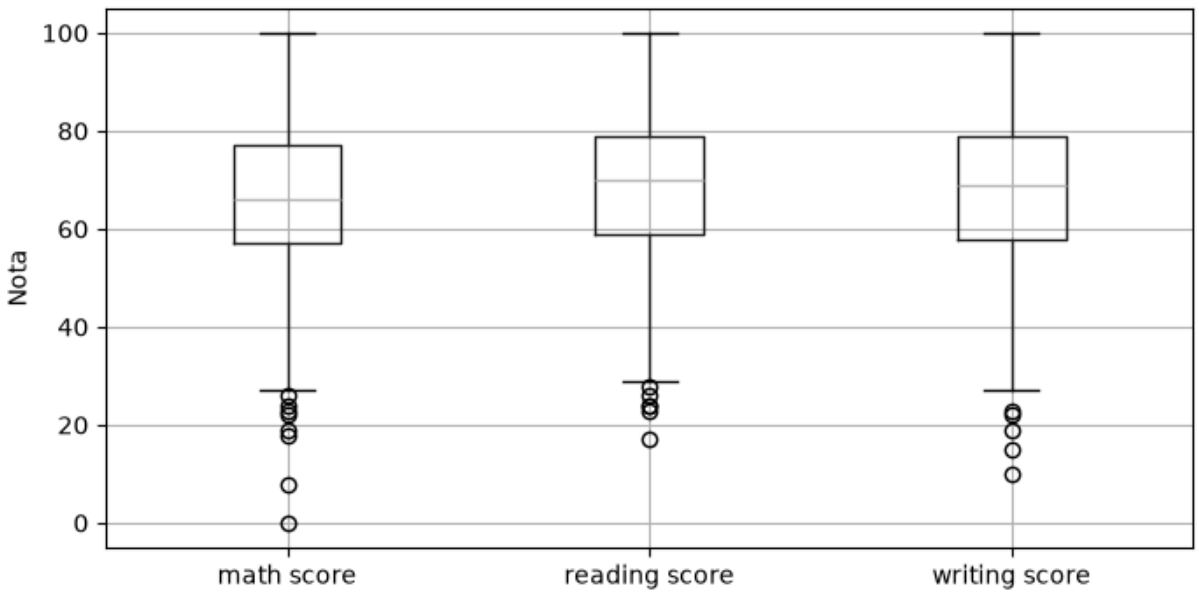
Ejercicio 2.4 — Solución

In [9]:
```
cols = ["math score", "reading score", "writing score"]

df[cols].boxplot(figsize=(8, 4))
plt.ylabel("Nota")
plt.show()

for col in cols:
    q1, q3 = df[col].quantile([0.25, 0.75])
    limite = q1 - 1.5 * (q3 - q1)
    print(f"{col}: límite inferior = {limite:.1f}, atípicos por debajo = {(df[col]

df[df["math score"] <= 10]
```



```
math score: límite inferior = 27.0, atípicos por debajo = 8
reading score: límite inferior = 29.0, atípicos por debajo = 6
writing score: límite inferior = 25.9, atípicos por debajo = 5
```

Out[9]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
59	female	group C	some high school	free/reduced	none	0	17	10
980	female	group B	high school	free/reduced	none	8	24	23

> **Respuesta explicada**
>
> - Hay atípicos **solo por debajo**, ninguno por encima de 100: `math` 8 (límite 27), `reading` 6 (límite 29) y `writing` 5 (límite 25.9). La cola izquierda de los histogramas aparece aquí como puntos aislados.
> - Los estudiantes con `math` ≤ 10 son dos: uno con 0 (reading 17, writing 10) y otro con 8 (reading 24, writing 23). Ambos son `female` con `lunch = free/reduced`.
> - **Error común:** borrar atípicos automáticamente. Aquí los dos casos también tienen notas muy bajas en las otras materias, lo que apunta a un desempeño realmente bajo y no a un dato mal digitado; no hay evidencia para eliminarlos.

Respuesta esperada — Pregunta de reflexión 5: `math` tiene más atípicos (8), todos hacia valores bajos. El 0 es plausible como nota real porque el mismo estudiante tiene las notas más bajas de lectura y escritura (17 y 10); un error de captura aislado no tendría esa coherencia. Lo razonable es **conservarlo** y, si se quiere, reportar los estadísticos con y sin él. Como la mediana resiste los extremos, es mejor resumen que la media cuando hay atípicos.

Parte 3 — Reto integrador (opcional) — Solución de referencia

In [10]:
```
df.groupby("lunch")["math score"].agg(["mean", "median", "std", "count"]).round(2)
```

Out[10]:

lunch	mean	median	std	count
free/reduced	58.92	60.0	15.16	355
standard	70.03	69.0	13.65	645

o alternativamente:

In [11]:
```
df.groupby("parental level of education")["math score"].agg(["mean", "median", "std
```

Out[11]:

parental level of education	mean	median	std	count
master's degree	69.75	73.0	15.15	59
bachelor's degree	69.39	68.0	14.94	118
associate's degree	67.88	67.0	15.11	222
some college	67.13	67.5	14.31	226
some high school	63.50	65.0	15.93	179
high school	62.14	63.0	14.54	196

> **Respuesta explicada**
>
> - `lunch` : `standard` 70.03 (mediana 69, n = 645) frente a `free/reduced` 58.92 (mediana 60, n = 355). La diferencia de ~11 puntos es amplia frente a la desviación estándar de cada grupo (13.65 y 15.16), y ambos grupos son lo bastante grandes como para fiarse de ella.
> - Con `count` el estudiante ve que el promedio de cada categoría no tiene el mismo respaldo: es el vínculo con la reflexión 1.
> - Con `parental level of education`, ordenado por media, los dos niveles más bajos (`high school` 62.14 y `some high school` 63.50) quedan por debajo de los demás y `master's degree` (69.75) es el más alto. Ojo: `high school` rinde menos que `some high school`, así que la relación no es estrictamente ordenada.

Respuesta esperada — Pregunta de cierre: El grupo `lunch = "standard"` tiene un promedio de `math score` más alto (70.03) que `free/reduced` (58.92): unos 11 puntos de diferencia, cerca de 0.7 desviaciones estándar. Por `parental level of education` el promedio sube de 62.14 (`high school`) a 69.75 (`master's degree`), unos 7.6 puntos. Es una buena oportunidad para introducir la diferencia entre **correlación y causalidad**: el dataset no permite concluir que el tipo de almuerzo *cause* el resultado, puede reflejar factores socioeconómicos subyacentes no capturados directamente en esta variable.

> *Cierre sugerido de la sesión:* recalcar que este taller cubre el análisis **descriptivo** (qué pasó, cómo se distribuyen los datos) — no predictivo ni causal — y que ese es el paso previo indispensable antes de cualquier modelo.

Rúbrica rápida de participación (sugerida)

Criterio	Logrado
Inspecciona el dataset (`shape` , `info` , nulos, duplicados) antes de analizar	☐
Calcula frecuencias y porcentajes correctamente para 2 variables categóricas	☐
Interpreta correctamente el balance/desbalance de categorías	☐
Obtiene `describe()` e interpreta media vs. mediana	☐
Genera histogramas y reconoce (o descarta) la forma de campana	☐
Completa el reto integrador con `groupby` y propone una hipótesis razonable	☐
Usa el boxplot para identificar atípicos y decide con argumentos si conservarlos	☐